Namaa Community at IslamicEval 2026: Retrieval-Grounded Verification of Qur'anic and Hadith Citations for Hallucination Identification

Fatimah Emad Eldin[1] **Israa Elhosiny**[2] **Khloud Al Jallad**[3] **Omer Nacar**[4]

[1]Trouve Labs [2]NAMAA Community [3]Arab International University [4]Tuwaiq Academy

fatimah@trouve.works israaelhosiny@gmail.com

k-aljallad@aiu.edu.sy o.najar@tuwaiq.edu.sa

Abstract

Large language models may generate inaccurate Islamic citations when answering religious questions. We present the Namaa Community system for Subtask 2 of IslamicEval 2026, which identifies hallucinations in Qur'anic verses (Ayah), hadith texts (matn), chains of narration (isnad), and claimed sources. We formulate the task as retrieval-grounded verification: citation spans are normalized and matched against the Qur'an and the six major hadith collections using character n-gram retrieval and edit-distance re-ranking. Type-specific verifiers assess Ayah and matn similarity, while claimed sources and isnads are checked against their parent citations. The deterministic pipeline requires no language model or GPU, and every decision is traceable to a canonical record. It achieves a macro accuracy of 0.846 on the development set, 17.9 percentage points above a per-type majority-class baseline, and 0.668 on the official blind test; we trace most of that shortfall to a submission-time configuration error and decompose the remainder by segment type. Our code and submissions are publicly available.[1]

1 Introduction

When large language models answer religious questions they frequently quote scripture—a verse of the Qur'an or a saying of the Prophet (a hadith)—and such quotations are a distinctive locus of hallucination: a model may alter a verse, misattribute a saying, or invent a chain of narrators. Unlike open-domain factuality, the ground truth here is finite and canonical, so faithfulness can be checked exactly. IslamicEval 2026 (Alharbi et al., 2026) formalises this over Arabic responses, continuing the inaugural edition (Mubarak et al., 2025).

Our team entered both subtasks; this paper describes the Subtask 2 system, in which the citation spans are given and the system returns a verdict for each. Every span is one of four types: a quoted verse (Ayah), a hadith body (matn), a chain of narrators (isnad), or a stated attribution (claimed source). Systems are ranked by accuracy computed separately within each type and then macro-averaged over the four (Alharbi et al., 2026), so a rare type weighs exactly as much as a frequent one.

We therefore give each type its own verifier rather than optimising only the two abundant text types: a verse must reproduce its source almost word for word, a hadith body is allowed the variation transmission introduces, and each dependent segment is checked against its parent. Because the evidence is a closed canon, no language model is used to reach a verdict—a transparent match against a named canonical record is cheaper than a neural judgement and, for religious content, easier to defend.

Isnad and claimed source are the two types that depend on a parent text segment, and a verifier that ignores that parent handles both badly. Making them parent-aware raises development macro accuracy from 0.725 to 0.846 (Table 1), since the macro metric gives these two rare types half its weight. The official blind test scores 0.668; we report that figure unchanged, trace most of the shortfall to a submission-time error (Section 4.2), and decompose the remainder by type (Section 4.3).

2 Background

Task setup. For each generated Arabic response, Subtask 2 supplies the citation spans already located by the organisers, together with the type of each. The input is therefore a response paired with a typed span, and the output a binary verdict—correct or incorrect—for every span. Of the four types, Ayah and matn are text segments; isnad and claimed source are structurally dependent, and are scored only when their parent text segment is correct. Otherwise they count as non-applicable and the official

[1]https://github.com/NAMAA-ORG/Namaa-Community-at-IslamicEval-2026

Proceedings of The Fourth Arabic Natural Language Processing Conference (ArabicNLP 2026) - Volume 2 Shared Tasks, pages 1098–1105
October 28-29, 2026 ©2026 Association for Computational Linguistics

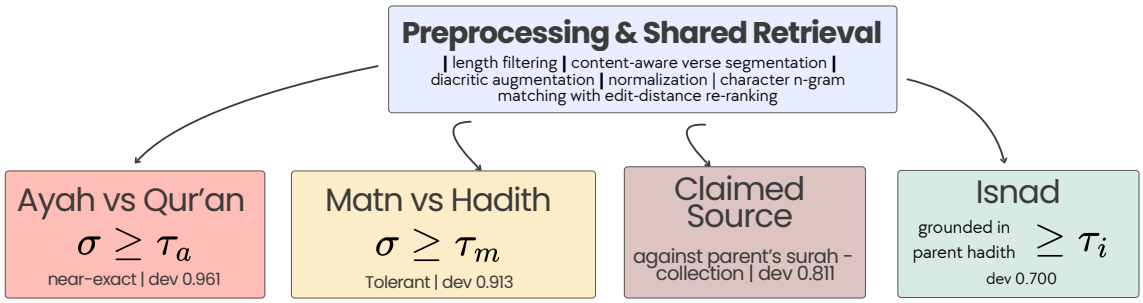


Figure 1: The Namaa Community pipeline. A shared preprocessing and retrieval stage grounds every segment in the canonical corpora; four typed verifiers, one per segment type, produce the verdict. Development accuracy is given beside each verifier (macro 0.846).

scorer drops them.

Data. Verification is grounded against the two corpora the organisers provide, the canonical Qur'an and the six major hadith collections. The Qur'an comprises 6,236 verses and the hadith corpus 34,994 records, of which 31,811 carry a non-empty body; each record also stores its complete narration, chain and body together, which the isnad verifier uses for grounding (Section 3). The development split has 484 responses and 2,728 segments; after non-applicable rows are removed the scored segments number 698 Ayah (222 correct, 476 incorrect), 588 matn (121, 467), 429 claimed source (283, 146) and 30 isnad (16, 14). The text types are predominantly incorrect, the scored attributions predominantly correct, and the isnad is thin yet carries a full quarter of the metric.

Always predicting a type's more frequent label—the *majority-class fallback*, the behaviour any verifier is reduced to without evidence of its own—already reaches macro 0.667, the baseline row of Table 1. Because it is strong on the two skewed text types, the headroom the macro metric leaves lies almost entirely in the structural types. The blind-test split is released with its spans but without labels: 2,173 responses and 10,375 spans (3,539 Ayah, 3,076 matn, 3,295 claimed source, 465 isnad). Its scored supports still depend on the withheld labels, since a dependent segment is scored only when its parent is correct.

Related work. Verifying generated text against evidence is the concern of automated fact verification, from FEVER (Thorne et al., 2018) to reference-free hallucination detectors that score factual precision or self-consistency (Manakul et al., 2023; Min et al., 2023); for Arabic, it has been approached through stance over retrieved evidence (Alhindi et al., 2021). Our setting differs in one respect that shapes the whole design: the claims are exact quotations checked against fixed canonical sources, so verification reduces to grounded matching rather than entailment. The closest precedents are the inaugural IslamicEval systems (Mubarak et al., 2025): TCE (ElKoumy et al., 2025) and HUMAIN (Omayrah et al., 2025) report that Qur'anic quotations must match near word-for-word once diacritics are removed, while hadith bodies require tolerance—an asymmetry we adopt; BurhanAI (Al Adel et al., 2025) verifies through a layered exact-to-semantic index whose cheaper tiers we reuse; and our earlier entry (Emad Eldin, 2025) targeted span detection rather than verification.

3 System Overview

The system runs in two stages (Figure 1). A shared stage grounds each segment in the canonical corpora: it normalises the segment, retrieves candidate records, and scores the best match. A typed stage then applies the verifier belonging to that segment's type, which turns the match into a verdict. Nothing in the pipeline is trained. The only quantities fitted to data are three decision thresholds, and no neural model is evaluated at inference. We submitted a single configuration; the intermediate rows of Table 1 are ablations of it, not alternative submissions.

Preprocessing. Both corpora are prepared identically, so that an undiacritised quotation can align with a vocalised source. Records of extreme length are dropped as a guard on index size. Any remaining text longer than twenty-five sub-word tokens is split into at most two parts at the whitespace nearest its midpoint, so a partially quoted verse can

still match without a word being broken; the token count comes from the AraBERT tokenizer (Antoun et al., 2020), used purely as a length measure (Section 4.1). Every text keeps its vocalised original and gains a diacritic-free copy, and is then expanded into overlapping word windows, so a fragmentary quotation can match a window instead of failing against a whole verse.

A single normaliser is applied to corpus and query alike: it removes diacritics and the elongation mark, unifies the alef, ya, waw-hamza and ta-marbuta variants, and drops non-Arabic characters. Its diacritic ranges are specified by Unicode code point rather than by literal combining marks—a detail that looks cosmetic and is not (Appendix A).

Retrieval. Retrieval supplies candidates; it does not decide any verdict. For each corpus we build a TF-IDF index over word-bounded character n-grams of length three to five (Salton and Buckley, 1988; Pedregosa et al., 2011), since such subword units resist Arabic inflection better than whole words do. A query span is normalised, scored against that index, and the fifteen highest-scoring records are kept as a shortlist.

The shortlist is then re-ranked by two edit-distance measures (Levenshtein, 1966; Bachmann, 2021): a token-set ratio, which compares the texts as sets of words and so ignores word order, and a partial ratio, which aligns the query against the best-matching substring of the candidate. Each candidate takes the higher of the two, and we write $\sigma(x) \in [0, 1]$ for the score of the best candidate for span x. No similarity threshold acts in this stage: every span receives a shortlist, and thresholds enter only in the verifiers below.

Text segments. An Ayah or a matn is judged by comparing σ against a threshold: the verdict is correct when $\sigma(x) \geq \tau_t$ and incorrect otherwise, where $t \in \{a, m\}$ selects the threshold belonging to the type. The two thresholds differ, following the asymmetry noted above. A Qur'anic quotation is expected to reproduce its source almost exactly, so τ_a is set high; a hadith body tolerates the variation that transmission introduces, so τ_m is set lower.

Claimed source. An attribution names a location—a surah and verse number, or a hadith collection—rather than quoting scripture, so searching the corpora for its text is ill-posed. We reuse instead what the parent has already established: for each annotation we retain the record its parent Ayah or matn matched, and ask whether the surah and verse, or the collection, that the attribution names agrees with that record. This verifier applies no similarity threshold, since it compares identifying fields rather than text. It returns incorrect only where it finds a disagreement, and correct otherwise—including where the attribution names nothing it can resolve.

Isnad. An isnad is scored only when its parent matn is correct, and the verifier exploits that precondition: a correct matn has already matched a hadith record, and every record stores its complete narration, chain and body together. The chain to check against is therefore identified by the parent, never retrieved independently. We compute the similarity of the quoted chain to the complete narration of each of the parent's three strongest matches, take the highest, and return correct when it reaches τ_i. This gives the isnad verifier evidence of its own, so it is no longer reduced to the majority-class fallback of Section 2.

4 Experiments

4.1 Experimental Setup

Splits and fitting. The three decision thresholds are the only quantities fitted to data. They are selected on a sample of 1,200 responses from the training split, in two stages. First τ_a and τ_m are swept jointly over a grid and set to the pair that maximises macro accuracy. Then τ_i is swept and set to the value that maximises isnad accuracy, which is the only type it affects. The selected values, $\tau_a = 0.98$, $\tau_m = 0.94$ and $\tau_i = 0.85$, are frozen and applied unchanged to both evaluation splits. The development split is used for ablation and reporting only and never for fitting, so the development figures estimate generalisation rather than fit.

Thresholds and filters. Table 2 in Appendix A collects every threshold, cut-off and filter in the pipeline and marks which are fitted. Only the three verifier decision thresholds are fitted; every other value is fixed in advance. Retrieval applies no similarity threshold at all—it is a candidate generator, cut off by rank ($K = 15$) rather than by score—so a span whose candidates are all poor is not filtered out but passed to its verifier, which judges it incorrect on the low σ. The claimed-source verifier uses no threshold either, since it compares identifying fields rather than text.

Implementation. The pipeline is Python and runs on CPU. Candidate generation uses the scikit-learn TF-IDF vectoriser (Pedregosa et al., 2011) with a word-bounded character-n-gram analyser ($n \in [3, 5]$); re-ranking uses the token-set and partial ratios of RapidFuzz (Bachmann, 2021), normalised to $[0, 1]$. Package versions are pinned in the released environment specification.[2]

How AraBERT is used. AraBERT (Antoun et al., 2020) enters the pipeline at exactly one point, and not as a model: its tokenizer measures the length of a text in sub-word tokens. That measurement drives the two length-based preprocessing rules of Section 3—dropping records of extreme length, and splitting text longer than twenty-five sub-word tokens—and nothing else. No weights are loaded, no embedding is computed, no component is fine-tuned, and AraBERT contributes to no similarity score and no verdict; substituting a whitespace word count would change only how those two rules count. The system runs no neural forward pass at inference, which is why it needs no GPU. The released code does contain an optional AraBERT pair-classifier notebook, but that is exploratory: it is not part of the submitted system and contributes to none of the numbers reported here.

Decoding, post-processing and metric. There is no decoding step: each verdict is a comparison against a fixed threshold or a field-level agreement test, so the system is deterministic once the corpus index is fixed. A verdict is emitted for every segment, including those the scorer discards as non-applicable, so no prediction is ever missing. Every reported figure comes from the organisers' official scorer, which computes accuracy separately within each segment type and macro-averages over the four.

4.2 Results

Table 1 reports the development ablation together with the official blind-test submission. The ablation adds one component at a time. The initial configuration matches the attribution against the corpus as quoted text and leaves the isnad to the majority-class fallback, reaching macro 0.725. Linking the attribution to its parent record instead raises that type from 0.492 to 0.811, and the macro average by 8.0 points. Grounding the isnad in the parent hadith's narration then raises the isnad from 0.533 to 0.700, and the macro by a further 4.2 points, to 0.846, which is 12.2 points above the initial configuration and 17.9 above the fallback baseline. Neither step changes either text type, so the whole improvement is carried by the structural types, which is exactly what the macro metric rewards. Appendix B shows the result is insensitive to the choice of retrieval backend.

Accounting for the development–test gap. The official blind-test macro of 0.668 lies 17.8 points below the development estimate. Two factors contribute, and they differ in how far each can be substantiated.

The first concerns the submission itself: the file scored by the organisers was not generated by the configuration of the last row of Table 1. We record this because it materially qualifies the official figures, while acknowledging that it cannot be confirmed from the released artefacts.

The second is a shift in the test inputs, and unlike the first it is directly measurable without the withheld labels. Applying the final configuration to the released test spans, the proportion it labels *correct* roughly doubles against development: 0.345 to 0.557 for Ayah, 0.241 to 0.440 for matn, and 0.300 to 0.636 for claimed source (Appendix B). Test citations agree with their canonical sources considerably more often than development ones do. Since the thresholds were fitted where citations are predominantly incorrect—on development, 0.318 of scored Ayah and 0.206 of scored matn are correct—they operate at test time at a point they were not calibrated for.

The two factors are complementary rather than alternative, and the claimed source implicates both. Under the shift alone, a verifier that returns correct unless it detects a mismatch should *gain* accuracy on a test set whose citations agree with their sources more often, not fall to 0.340. That the opposite occurred is consistent with the parent linkage having been disrupted in the scored file, though the withheld labels preclude a definitive attribution. We therefore report the official figures unchanged rather than substitute re-scored ones: the gap is not evidence that retrieval-grounded verification fails to transfer, but neither can we settle its cause from what was released.

4.3 Error Analysis

The blind-test profile is uneven rather than uniformly depressed. The Ayah verifier holds at 0.818,

[2] https://github.com/NAMAA-ORG/Namaa-Community-at-IslamicEval-2026

Configuration	Ayah	matn	c. src	isnad	Macro
Development set					
Majority-class fallback	.682	.794	.660	.533	.667
Attribution as text	.961	.913	.492	.533	.725
+ parent-link	.961	.913	.811	.533	.805
+ grounded isnad	.961	.913	.811	**.700**	**.846**
Official blind test					
Submitted system	.818	.622	.340	.895	.668

Table 1: Per-segment-type accuracy and macro average; leading zeros omitted, "c. src" is the claimed source. Upper panel: development, starting from the per-type majority-class fallback, each row then adding one component to the row above. Lower panel: the official score of the submitted file, which did not reflect the last development row's configuration (Section 4.2); its scored per-type supports depend on the withheld labels.

so near-exact Qur'anic matching clearly transfers. The isnad reaches 0.895, above its development value, but we decline to read that as evidence that grounding generalises well: the development estimate rests on thirty scored instances, and while the test split has 465 isnad spans, how many were scored depends on the withheld labels.

The matn falls from 0.913 to 0.622, the decline the input shift of Section 4.2 predicts most directly: one fixed τ_m cannot separate the classes at two operating points. The claimed source is treated there as well.

One property cuts across all four types: the errors are coupled by construction. A single retrieval miss on a parent Ayah or matn propagates to both of its dependent segments, so the structural types can never be more reliable than the text types they hang from—the price of reusing the parent match, paid where the macro metric is most sensitive. Appendix C illustrates both what the structural verifiers buy and one misclassification per type.

5 Conclusion

The Namaa Community system verifies Islamic citations by grounding them in canonical sources rather than by asking a language model to judge them. Every verdict is a comparison against a named canonical record and the pipeline runs on CPU, so the result is auditable in a way a neural judgement is not—for religious content a substantive property, not a convenience. Verifying the two rare structural types against their parent citations, instead of in isolation, carries development macro accuracy from 0.725 to 0.846.

The blind-test score of 0.668 is considerably lower, and we attribute it to a submission error together with a measurable shift in the test inputs (Section 4.2). Both point the same way: thresholds that adapt to the operating point rather than one global τ_m, and an attribution verifier that does not inherit its parent's retrieval errors wholesale.

Limitations

Verification here is lexical. Similarity is character n-gram retrieval refined by edit distance, so a hadith transmitted with different wording but the same meaning falls below τ_m and is judged incorrect. Recognising that kind of paraphrase would need semantic matching, which this pipeline deliberately does not perform, and which would cost the auditability that motivates it.

The thresholds are global and frozen after fitting. Section 4.2 shows test spans agreeing with their sources roughly twice as often as development spans, and one threshold per type cannot serve both operating points; the system neither detects such a shift nor adapts to it.

The two structural verifiers inherit their parent's errors by construction, so a retrieval miss on an Ayah or a matn propagates to its claimed source and its isnad. The claimed-source verifier also returns correct whenever it cannot resolve the reference it is given—when a surah is cited by number rather than by name, for instance—so it under-detects rather than over-detects.

Our isnad evidence is thin: thirty scored development instances, and blind-test labels that were never released, so neither the isnad result nor the corrected configuration can be independently verified (Section 4.3). Coverage is bounded by the two corpora the organisers supply: an attribution to a collection outside the six major books cannot be checked at all. The system is built for Arabic only.

Acknowledgments

We thank the IslamicEval 2026 organisers for the data, the grounding corpora, and the evaluation infrastructure, and the anonymous reviewers for their comments. We also thank Tuwaiq Academy for their support.

References

Arij Al Adel, Abu Bakr Soliman, Mohamed Sakher Sawan, Rahaf Al-Najjar, and Sameh Amin. 2025. BurhanAI at IslamicEval 2025 shared task: Combating hallucinations in LLMs for Islamic content; evaluation, correction, and retrieval-based solution.

In *Proceedings of the Third Arabic Natural Language Processing Conference (ArabicNLP): Shared Tasks*. Association for Computational Linguistics.

Rahaf Alharbi, Abdulelah Alturki, Watheq Mansour, Rana Malhas, Hamdy Mubarak, Kareem Darwish, Tamer Elsayed, and Walid Magdy. 2026. IslamicEval 2026: The second shared task of capturing LLMs hallucination in Islamic content. In *Proceedings of the Fourth Arabic Natural Language Processing Conference (ArabicNLP 2026)*.

Tariq Alhindi, Amal Alabdulkarim, Ali Alshehri, Muhammad Abdul-Mageed, and Preslav Nakov. 2021. AraStance: A multi-country and multi-domain dataset of Arabic stance detection for fact checking. In *Proceedings of the Fourth Workshop on NLP for Internet Freedom (NLP4IF)*.

Wissam Antoun, Fady Baly, and Hazem Hajj. 2020. AraBERT: Transformer-based model for Arabic language understanding. In *Proceedings of the 4th Workshop on Open-Source Arabic Corpora and Processing Tools*, pages 9–15.

Max Bachmann. 2021. RapidFuzz: Rapid fuzzy string matching in Python.

Mohammed ElKoumy, Khalid Allam, Ahmed Tamer, and Mohammed Elqabalawy. 2025. TCE at IslamicEval 2025: Retrieval-augmented LLMs for quranic and hadith content identification and verification. In *Proceedings of the Third Arabic Natural Language Processing Conference (ArabicNLP): Shared Tasks*. Association for Computational Linguistics.

Fatimah Emad Eldin. 2025. Isnad AI at IslamicEval 2025: A rule-based system for identifying islamic citation in LLM outputs. In *Proceedings of the Third Arabic Natural Language Processing Conference (ArabicNLP): Shared Tasks*, pages 540–559. Association for Computational Linguistics.

Vladimir I. Levenshtein. 1966. Binary codes capable of correcting deletions, insertions, and reversals. *Soviet Physics Doklady*, 10(8):707–710.

Potsawee Manakul, Adian Liusie, and Mark J. F. Gales. 2023. SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 9004–9017.

Sewon Min, Kalpesh Krishna, Xinxi Lyu, Mike Lewis, Wen-tau Yih, Pang Wei Koh, Mohit Iyyer, Luke Zettlemoyer, and Hannaneh Hajishirzi. 2023. FActScore: Fine-grained atomic evaluation of factual precision in long form text generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 12076–12100.

Hamdy Mubarak, Rana Malhas, Watheq Mansour, Abubakr Mohamed, Mahmoud Fawzi, Majd Hawasly, Tamer Elsayed, Kareem Darwish, and Walid Magdy. 2025. IslamicEval 2025: The first shared task of capturing LLMs hallucination in Islamic content. In *Proceedings of the Third Arabic Natural Language Processing Conference (ArabicNLP): Shared Tasks*, pages 480–493. Association for Computational Linguistics.

Arwa Omayrah, Sakhar Alkhereyf, Ahmed Abdelali, Abdulmohsen Althubaity, Jeril Kuriakose, and Ibrahim AbdulMajeed. 2025. HUMAIN at IslamicEval 2025 shared task 1: A three-stage LLM-based pipeline for detecting and correcting hallucinations in quran and hadith. In *Proceedings of the Third Arabic Natural Language Processing Conference (ArabicNLP): Shared Tasks*. Association for Computational Linguistics.

Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, et al. 2011. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830.

Gerard Salton and Christopher Buckley. 1988. Term-weighting approaches in automatic text retrieval. *Information Processing & Management*, 24(5):513–523.

James Thorne, Andreas Vlachos, Christos Christodoulopoulos, and Arpit Mittal. 2018. FEVER: a large-scale dataset for fact extraction and VERification. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL-HLT)*, pages 809–819.

A Implementation Details

The retriever indexes character n-grams of length three to five and returns a fifteen-candidate shortlist, which is re-ranked by the higher of a token-set and a partial edit-distance ratio, normalised to $[0, 1]$. The three thresholds, fitted on a 1,200-response training sample, are $\tau_a = 0.98$, $\tau_m = 0.94$ and $\tau_i = 0.85$, and isnad grounding scores the three strongest parent-matn matches. Table 2 lists these values together with every other cut-off in the pipeline, and Table 3 illustrates the preprocessing steps of Section 3; the original text is always retained alongside any derived form.

One implementation detail is invisible on inspection and yet fatal. The normaliser's diacritic ranges must be written numerically, by Unicode code point, and not by typing the Arabic combining marks literally. Literal marks do not render as standalone glyphs, and they can reorder relative to a range delimiter when a file is saved, which silently widens the range to cover the base letters. The normaliser then deletes all Arabic text, and every span fails to match.

Stage	Quantity	Value	Fitted
Preproc.	length filter (chars)	1500	no
Preproc.	split above (tokens)	25	no
Preproc.	window length (words)	5–15	no
Preproc.	window step (words)	3	no
Retrieval	char. n-gram range	3–5	no
Retrieval	shortlist size K	15	no
Retrieval	similarity cut-off	none	—
Ayah	threshold τ_a	0.98	yes
matn	threshold τ_m	0.94	yes
isnad	threshold τ_i	0.85	yes
isnad	parent matches scored	3	no
c. src	threshold	none	—

Table 2: Every threshold, cut-off and filter in the pipeline. Only the three verifier decision thresholds are fitted, on a 1,200-response training sample; all other values are fixed in advance. Retrieval and the claimed-source verifier apply no threshold: retrieval is cut off by rank, and the claimed-source verifier compares identifying fields rather than text.

Transformation	Illustration
Segmentation of over-length verses	a long verse is divided into two parts at the whitespace nearest its midpoint, with no word broken
Diacritic augmentation	the vocalised original is kept and an undiacritised copy added, e.g. الحمد لله رب العالمين alongside its fully marked form
Overlapping windows	a twenty-word body yields windows of five to fifteen words over both the original and normalised forms

Table 3: Preprocessing transformations.

B Component Ablations

Grounding the quoted chain in the parent hadith's complete narration beats the majority-class fallback by nearly seventeen points on development (Table 4). Grounding it in the chain portion alone ties that on development, but is six points weaker on the training sample—and the training sample is the one we selected on, development never having been used for fitting. The similarity itself separates the two classes, averaging 0.84 for correct chains against 0.75 for incorrect ones.

Isnad verifier	Train acc.	Dev acc.
Majority-class fallback	—	0.533
Grounded in chain only	0.663	0.700
Grounded in full narration	0.719	0.700

Table 4: Isnad verification strategies.

Development-to-test input shift. Table 5 reports, for the final configuration, the share of spans of each type that it labels *correct* on development and on the released blind-test spans. The measurement needs no labels, and it shows the two splits presenting the verifiers with materially different inputs.

Segment type	Dev	Test	Spans (dev / test)
Ayah	.345	.557	698 / 3,539
matn	.241	.440	588 / 3,076
isnad	.328	.484	204 / 465
c. src	.300	.636	1,238 / 3,295

Table 5: Share of spans the final configuration labels *correct*, over all spans of each type rather than only the scored ones, on development and on the released blind-test spans. Computed without gold labels; leading zeros omitted.

Table 6 isolates the candidate retriever. Swapping the character n-gram index for word-level TF-IDF or for Okapi BM25, with every other component held fixed, moves macro accuracy by at most 0.005, and BM25 is in fact marginally stronger on both text types. We read this as support for the paper's main claim rather than as a defence of our own choice: candidate retrieval is not the binding constraint here, the typed verifiers are.

Backend	Ayah	matn	c. src	isnad	Macro
char. n-gram (ours)	.961	.913	.811	.700	**.846**
word-level TF-IDF	.961	.912	.823	.667	.841
Okapi BM25	.963	.927	.823	.667	.845

Table 6: Development accuracy with only the retriever swapped; leading zeros omitted. Retriever choice is not what separates configurations; the typed verifiers are.

C Qualitative Examples

Two cases show what the structural verifiers buy. In the first, a hadith body is quoted correctly but attributed to البخاري, while the record it matched belongs to مسلم: the parent-linked verifier catches the collection mismatch and returns incorrect even though the body itself is authentic. In the second, a chain whose similarity to the parent hadith's narration reaches 0.90 clears τ_i, whereas a chain naming transmitters absent from that narration falls below it. Neither distinction is available to a verifier that can only predict the majority class.

Table 7 turns to failures, giving one representative misclassification per segment type with the quoted span, the gold and predicted labels, and the nearest source retrieved. Each illustrates a different limit. The Ayah case differs from its source by one diacritic, which normalisation removes by design, so the verifier cannot see the alteration at all. The matn case quotes only part of a longer record

and falls below τ_m. The isnad case names a single narrator in a span far shorter than the narration it is scored against. The claimed-source case gives the surah numerically rather than by name, so the verifier resolves no surah, detects no mismatch, and returns correct—the default behaviour described in Section 3, here producing the wrong answer.

Type **gold/pred**	**Span / nearest source**
Ayah inc/cor	وَقَالَ رَبُّكُمْ ادْعُونِي أَسْتَجِبْ لَكُمْ وَقَالَ رَبُّكُمُ ادْعُونِي أَسْتَجِبْ لَكُمْ إِنَّ ا...
matn cor/inc	إن الله يرضى لكم ثلاثًا: أن تعبدوه ولا تشركوا به شيئًا... إِنَّ اللَّهَ يَرْضَى لَكُمْ ثَلَاثًا ، وَيَكْرَهُ لَكُمْ...
isnad cor/inc	عن علي رضي الله عنه قال: كُنْتُ رَجُلًا مَذَّاءً ، وَكُنْتُ أَسْتَحْيِي أَنْ أَس...
c. src inc/cor	السورة 3، آية 139 فَإِذَا بَلَغْنَ أَجَلَهُنَّ فَأَمْسِكُوهُنَّ بِمَعْرُو...

Table 7: Development misclassifications, one per type: quoted span (upper) against nearest retrieved source (lower). Labels gold/pred, abbreviated inc/cor; "c. src" is the claimed source.